

Проектирование информационной архитектуры и взаимодействия с пользователем

Лабораторная работа №4

Щербакова Полина, Игнатенок Дарья, Павлова Анна
4 курс группы 12-13

Задание 1. Проектирование модели требований и реестра вариантов использования

1. Изучить статью «[Проектирование REST API на примере интернет-магазина](#)», материал которой используется в качестве основы для реализации последующих заданий.
2. На основе требований из лабораторной 3 описать требования в виде модели (см. рис. 1 ниже): Действующее лицо → User Story → Use Case

Актор	User Story	Use Case
Покупатель	US-1.1 Как покупатель интернет-магазина, я хочу видеть перечень товаров, чтобы узнать подробности ассортимента (название товара, поставщик, стоимость, количество единиц в наличии)	UC 1.1 Посмотреть список товаров
Покупатель	US-1.2 Как покупатель интернет-магазина, я хочу видеть перечень поставщиков, чтобы узнать подробности о продавцах (название компании, телефон, емейл, адрес)	UC 1.2 Посмотреть список поставщиков
Менеджер	US-2.1.1 Как менеджер интернет-магазина, я хочу видеть перечень товаров с их данными (название, поставщик, стоимость, количество единиц в наличии), чтобы повысить эффективность товарного ассортимента	UC 1.1 Посмотреть список товаров

Актор	User Story	Use Case
Клиент	US-1.1 Как клиент, я хочу просматривать меню ресторана, чтобы выбрать блюда.	UC 1.1 Посмотреть меню
Клиент	US-1.2 Как клиент, я хочу просматривать отзывы о еде и обслуживании, чтобы иметь представление о заведении.	UC 1.2 Посмотреть отзывы
Клиент	US-1.3 Как клиент, я хочу оставлять отзывы о еде и обслуживании, чтобы поделиться своим опытом.	UC 1.3 Оставить отзыв

Клиент	US-1.4 Как клиент, я хочу знать об акциях в заведениях, чтобы заказывать выгодно.	UC 1.4 Посмотреть акции
Клиент	US-1.5 Как клиент, я хочу просматривать свою историю заказов, чтобы при необходимости повторить.	UC 1.5 Посмотреть историю заказов
Клиент	US-1.6 Как клиент, я хочу добавлять блюда в корзину, чтобы сделать заказ.	UC 1.6 Добавить блюдо в корзину UC 1.7 Сделать заказ
Клиент	US-1.7 Как клиент, я хочу забронировать место в ресторане.	UC 1.8 Забронировать место
Администратор	US-2.1 Как администратор, я хочу просматривать меню ресторана.	UC 1.1 Посмотреть меню
Администратор	US-2.2 Как администратор ресторана, я хочу управлять меню (добавлять, изменять, удалять блюда), чтобы поддерживать актуальность предложения.	UC 2.1 Добавить блюдо UC 2.2 Изменить блюдо UC 2.3 Удалить блюдо
Администратор	US-2.3 Как администратор, я хочу просматривать отзывы о еде и обслуживании.	UC 1.2 Посмотреть отзывы
Администратор	US-2.4 Как администратор, я хочу просматривать список заказов, чтобы оценивать эффективность заведения по состояниям заказов.	UC 2.4 Посмотреть список заказов
Администратор	US-2.5 Как администратор, я хочу изменять состояние клиентского заказа, чтобы информировать клиента о его готовности.	UC 2.5 Посмотреть заказ UC 2.6 Изменить состояние заказа

Администратор	US-2.6 Как администратор, я хочу подтверждать бронь, чтобы контролировать поток клиентов.	UC 2.4 Подтвердить бронь
---------------	---	--------------------------

3. Добавить модель требований и реестр вариантов использования в документацию проекта в вики и github pages.

Актор	User Story	Use Case
Клиент	US-1.1 Как клиент, я хочу просматривать меню ресторана, чтобы выбрать блюда.	UC 1.1 Посмотреть меню
Клиент	US-1.2 Как клиент, я хочу просматривать отзывы о еде и обслуживании, чтобы иметь представление о заведении.	UC 1.2 Посмотреть отзывы
Клиент	US-1.3 Как клиент, я хочу оставлять отзывы о еде и обслуживании, чтобы поделиться своим опытом.	UC 1.3 Оставить отзыв
Клиент	US-1.4 Как клиент, я хочу знать об акциях в заведениях, чтобы заказывать выгодно.	UC 1.4 Посмотреть акции
Клиент	US-1.5 Как клиент, я хочу просматривать свою историю заказов, чтобы при необходимости повторить.	UC 1.5 Посмотреть историю заказов
Клиент	US-1.6 Как клиент, я хочу добавлять блюда в корзину, чтобы сделать заказ.	UC 1.6 Добавить блюдо в корзину UC 1.7 Сделать заказ
Клиент	US-1.7 Как клиент, я хочу забронировать место в ресторане.	UC 1.8 Забронировать место
Администратор	US-2.1 Как администратор, я хочу просматривать меню ресторана.	UC 1.1 Посмотреть меню
Администратор	US-2.2 Как администратор ресторана, я хочу управлять меню, чтобы поддерживать актуальность предложения.	UC 2.1 Добавить блюдо UC 2.2 Изменить блюдо UC 2.3 Удалить блюдо
Администратор	US-2.3 Как администратор, я хочу просматривать отзывы о еде и обслуживании.	UC 1.2 Посмотреть отзывы
Администратор	US-2.4 Как администратор, я хочу просматривать список заказов, чтобы оценивать эффективность заведения.	UC 2.4 Посмотреть список заказов
Администратор	US-2.5 Как администратор, я хочу изменять состояние клиентского заказа, чтобы информировать клиента.	UC 2.5 Посмотреть заказ UC 2.6 Изменить состояние заказа
Администратор	US-2.6 Как администратор, я хочу подтверждать бронь, чтобы контролировать поток клиентов.	UC 2.4 Подтвердить бронь

Use Case Registry			
UC ID	Название	Основной(ые) актор(ы)	Краткое описание
UC 1.1	Посмотреть меню	Клиент, Администратор	Просмотр меню выбранного ресторана с перечнем блюд и цен.
UC 1.2	Посмотреть отзывы	Клиент, Администратор	Просмотр списка отзывов о еде и обслуживании в ресторане.
UC 1.3	Оставить отзыв	Клиент	Создание нового отзыва по завершённом визиту/заказу.
UC 1.4	Посмотреть акции	Клиент	Просмотр текущих акций и спецпредложений в заведениях.
UC 1.5	Посмотреть историю заказов	Клиент	Просмотр списка своих прошлых заказов с детализацией.
UC 1.6	Добавить блюдо в корзину	Клиент	Добавление выбранного блюда в корзину для последующего оформления заказа.
UC 1.7	Сделать заказ	Клиент	Оформление заказа на основе содержимого корзины.
UC 1.8	Забронировать место	Клиент	Отправка запроса на бронирование столика в ресторане.
UC 2.1	Добавить блюдо	Администратор	Добавление нового блюда в меню ресторана.
UC 2.2	Изменить блюдо	Администратор	Редактирование параметров существующего блюда (название, цена, описание и т.п.).
UC 2.3	Удалить блюдо	Администратор	Удаление блюда из меню ресторана.
UC 2.4	Посмотреть список заказов	Администратор	Просмотр списка заказов по ресторану с текущими статусами.
UC 2.5	Посмотреть заказ	Администратор	Просмотр деталей конкретного заказа клиента.
UC 2.6	Изменить состояние заказа	Администратор	Обновление статуса заказа (принят, готовится, готов, выполнен и т.п.).
UC 2.7	Подтвердить бронь	Администратор	Подтверждение запроса на бронирование столика или изменение статуса брони.

Задание 2. Проектирование физической модели базы данных

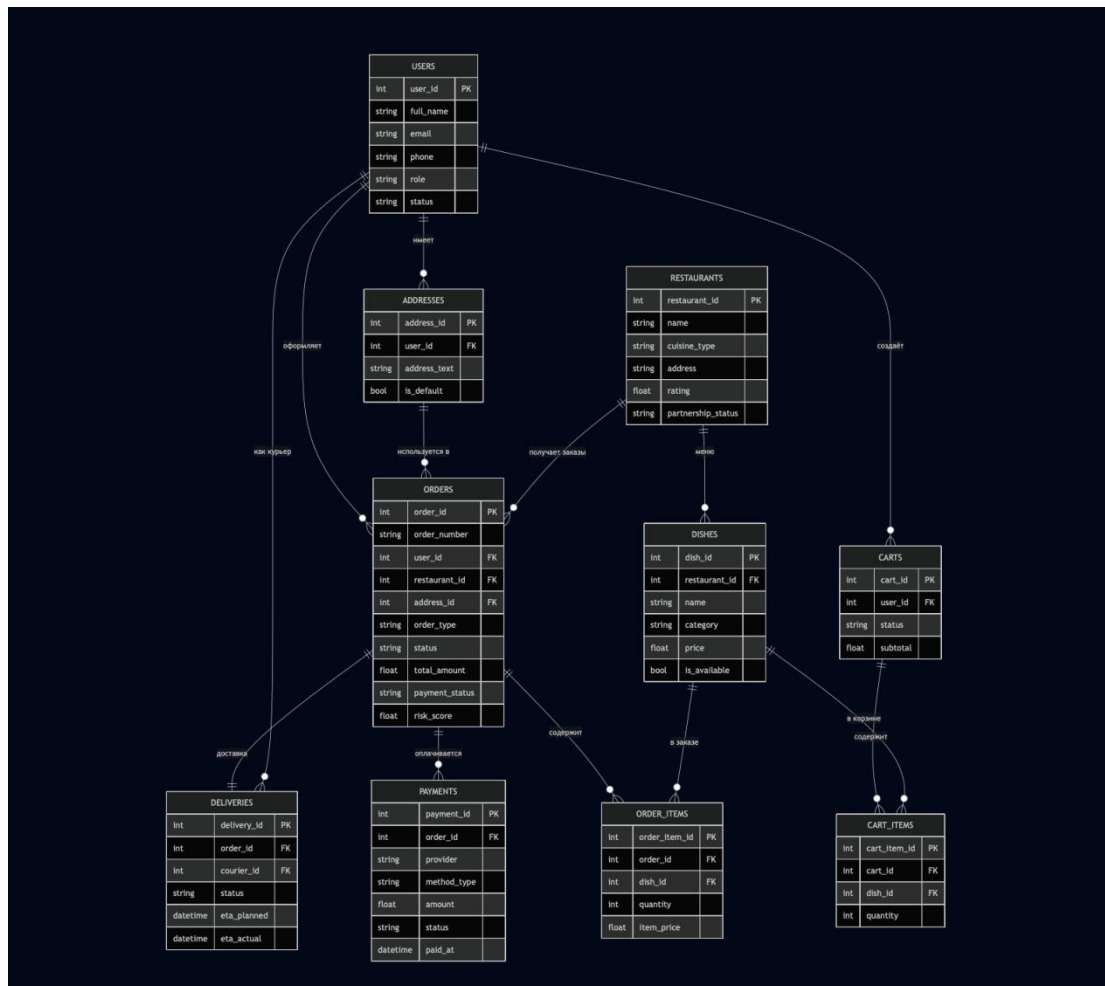
1. Уточнить логическую модель базы данных, созданную в 3-ей лаб. работе в виде ER-диаграммы, и представить в виде логической схемы вида (см. рис. 2):

Сущность	Таблица БД	Поле	Смысл поля	Тип данных
Товар	product	id	уникальный идентификатор товара	INTEGER
		name	название товара	VARCHAR(255)
		provider	уникальный идентификатор поставщика	INTEGER
		price	цена товара	
		quantity	количество товара на складе	INTEGER
Поставщик	provider	id	уникальный идентификатор поставщика	INTEGER
		name	название поставщика	VARCHAR(255)
		phone	контактный номер телефона поставщика	VARCHAR(255)
		email	электронная почта поставщика	VARCHAR(255)
		address	адрес поставщика	VARCHAR(255)

Рисунок 2

Объект	Зачем нужен	Ключевые атрибуты (остальное можно опустить/описать)
--------	-------------	--

		словами)
User	Все аккаунты: клиент, курьер, админ (роль в атрибуте, без отдельных сущностей)	user_id, ФИО, email, phone, role (customer/courier/admin), status
Address	Адреса доставки пользователя	address_id, user_id, строка адреса, is_default
Restaurant	Рестораны, из которых заказывают	restaurant_id, название, тип кухни, адрес, рейтинг, статус партнёрства
Dish	Блюда меню ресторана	dish_id, restaurant_id, название, категория, цена, признак доступности, краткое описание
Cart	Текущая корзина пользователя (для ЛР2)	cart_id, user_id, статус (active/abandoned), промежуточная сумма
CartItem	Позиции в корзине	cart_item_id, cart_id, dish_id, количество
Order	Оформленный заказ	order_id, order_number, user_id, restaurant_id, дата/время, статус, тип (доставка/самовывоз), адрес доставки (можно ссылкой на Address), total_amount, payment_status, краткий risk_score
OrderItem	Позиции в заказе	order_item_id, order_id, dish_id, количество, цена на момент заказа
Delivery	Информация о доставке	delivery_id, order_id, courier_id (ссылка на User с ролью курьер), статус, плановый/фактический eta
Payment	Оплата заказа	payment_id, order_id, провайдер, способ (карта/кошелёк), сумма, статус, время операции



2. Разработать физическую модель данных с учетом используемой СУБД в проекте и представить в виде диаграммы ER как на рисунке 3 и файла в формате .sql.

PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

-- 1. Пользователи

```

CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    role TEXT NOT NULL CHECK (role IN ('customer','courier','admin')),
    status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','blocked'))
);
  
```

-- 2. Адреса

```

CREATE TABLE addresses (
    address_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
  
```

```

    address_text TEXT NOT NULL,
    is_default INTEGER NOT NULL DEFAULT 0, -- 0/1 как bool
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- 3. Рестораны
CREATE TABLE restaurants (
    restaurant_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    cuisine_type TEXT,
    address TEXT,
    rating REAL NOT NULL DEFAULT 0.0,
    partnership_status TEXT NOT NULL DEFAULT 'active'
    CHECK (partnership_status IN ('active','paused','terminated'))
);

-- 4. Блюда
CREATE TABLE dishes (
    dish_id INTEGER PRIMARY KEY AUTOINCREMENT,
    restaurant_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    category TEXT,
    price REAL NOT NULL,
    is_available INTEGER NOT NULL DEFAULT 1, -- 1 = доступно
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

-- 5. Корзины
CREATE TABLE carts (
    cart_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','abandoned')),
    subtotal REAL NOT NULL DEFAULT 0.0,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- 6. Позиции в корзине
CREATE TABLE cart_items (
    cart_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
    cart_id INTEGER NOT NULL,
    dish_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    FOREIGN KEY (cart_id) REFERENCES carts(cart_id),
    FOREIGN KEY (dish_id) REFERENCES dishes(dish_id)
);

-- 7. Заказы
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

order_number TEXT NOT NULL UNIQUE,
user_id INTEGER NOT NULL,
restaurant_id INTEGER NOT NULL,
address_id INTEGER,
order_type TEXT NOT NULL CHECK (order_type IN ('delivery','pickup')),
status TEXT NOT NULL,
total_amount REAL NOT NULL DEFAULT 0.0,
payment_status TEXT NOT NULL DEFAULT 'unpaid'
CHECK (payment_status IN ('unpaid','paid','refunded')),
risk_score REAL NOT NULL DEFAULT 0.0,
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),
FOREIGN KEY (address_id) REFERENCES addresses(address_id)
);

```

-- 8. Позиции заказа

```

CREATE TABLE order_items (
order_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER NOT NULL,
dish_id INTEGER NOT NULL,
quantity INTEGER NOT NULL CHECK (quantity > 0),
item_price REAL NOT NULL,
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (dish_id) REFERENCES dishes(dish_id)
);

```

-- 9. Доставка

```

CREATE TABLE deliveries (
delivery_id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER NOT NULL UNIQUE, -- одна доставка на заказ
courier_id INTEGER NOT NULL, -- users.role = 'courier'
status TEXT NOT NULL,
eta_planned TEXT,
eta_actual TEXT,
FOREIGN KEY (order_id) REFERENCES orders(order_id),
FOREIGN KEY (courier_id) REFERENCES users(user_id)
);

```

-- 10. Платежи

```

CREATE TABLE payments (
payment_id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER NOT NULL,
provider TEXT,
method_type TEXT,
amount REAL NOT NULL,
status TEXT NOT NULL,
paid_at TEXT,
FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

```



```
CREATE INDEX idx_addresses_user ON addresses(user_id);
CREATE INDEX idx_dishes_restaurant ON dishes(restaurant_id);
CREATE INDEX idx_carts_user ON carts(user_id);
CREATE INDEX idx_cart_items_cart ON cart_items(cart_id);
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_orders_restaurant ON orders(restaurant_id);
CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_deliveries_courier ON deliveries(courier_id);
CREATE INDEX idx_payments_order ON payments(order_id);
```

```
polinaserbakova@MacBook-Air-Polina db_restohub % sqlite3 db_restohub.sqlite
SQLite version 3.43.2 2023-10-10 13:08:14
Enter ".help" for usage hints.
sqlite> .tables
addresses      carts           dishes          orders          restaurants
cart items     deliveries     order items     payments        users
```

Схема базы данных

ER-диаграмма (Mermaid)

```
erDiagram
    USER ||--o{ ADDRESS : "has"
    USER ||--o{ ORDER : "has"
    ADDRESS ||--o{ ORDER : "has"
    ORDER ||--o{ DELIVERY : "has"
    ORDER ||--o{ PAYMENT : "has"
    ORDER ||--o{ ORDER_ITEM : "has"
    ORDER ||--o{ CART : "has"
    ORDER_ITEM ||--o{ CART_ITEM : "has"
    RESTAURANT ||--o{ DISH : "has"
    DISH ||--o{ ORDER_ITEM : "has"
    DISH ||--o{ CART_ITEM : "has"
    CART ||--o{ CART_ITEM : "has"

    USER {
        int user_id PK
        string full_name
        string email
        string phone
        string role
        string status
    }
    ADDRESS {
        int address_id PK
        int user_id FK
        int restaurant_id FK
        string address_line
        bool is_default
    }
    ORDER {
        int order_id PK
        string order_number
        int user_id FK
        int restaurant_id FK
        int address_id FK
        string order_type
        string status
        float total_amount
        string payment_status
        float risk_score
    }
    DELIVERY {
        int delivery_id PK
        int order_id FK
        int courier_id FK
        string status
        datetime eta_planned
        datetime eta_actual
    }
    PAYMENT {
        int payment_id PK
        int order_id FK
        string provider
        string method_type
        float amount
        string status
        datetime paid_at
    }
    ORDER_ITEM {
        int order_item_id PK
        int order_id FK
        int dish_id FK
        int quantity
        float item_price
    }
    DISH {
        int dish_id PK
        int restaurant_id FK
        string name
        string category
        float price
        bool is_available
    }
    CART {
        int cart_id PK
        int user_id FK
        string status
        float subtotal
    }
    CART_ITEM {
        int cart_item_id PK
        int cart_id FK
        int order_item_id FK
        int quantity
    }
```

The diagram illustrates the database schema for a restaurant system. It includes tables for users, addresses, restaurants, dishes, orders, deliveries, payments, order items, and cart items. Relationships are defined between these tables, such as a user having multiple addresses and orders, an order being associated with a delivery and payment, and a cart containing multiple cart items.

Pages 8

Find a page...

- Home
- API проект
- Диаграмма файлов приложения
- Дизайн мобильного и веб-прило...
- Дополнительная спецификация
- Персонажи проекта RestoHub
- Схема базы данных ER-диаграмма (Mermaid)
- Функциональные требования

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/fpmi-hci-2026>

Задание 3. Проектирование маршрутов и конечных точек в виде спецификации API

1. На основе материалов статьи, приведенной в задании 1 выполнить проектирование, маршрутов и конечных точек и представить их в виде как на рисунке 4.

Сущность	Таблица БД	Маршрут (базовый)
Пользователь	users	/users
Авторизация (логин)	users	/login
Регистрация	users	/registration
Адрес пользователя	addresses	/addresses
Ресторан	restaurants	/restaurants
Блюдо	dishes	/dishes
Корзина	carts	/carts
Позиция корзины	cart_items	/cart-items
Заказ	orders	/orders
Позиция заказа	order_items	/order-items
Доставка	deliveries	/deliveries
Платёж	payments	/payments

2. Сопоставить варианты использования с конечными точками, т.е. HTTP-методами, которые будут обращаться к маршрутам как на рисунке 5:

Актор	Use Case	Маршрут	HTTP-запрос	Аутентификация
Пользователь (клиент)	UC 0 Войти в систему (форма логина)	/login	GET	нет
Пользователь (клиент)	UC 0.1 Аутентификация	/login	POST	нет
Пользователь (клиент)	UC 0.2 Зарегистрироваться (форма регистрации)	/registration	GET	нет
Пользователь (клиент)	UC 0.3 Отправить данные регистрации	/registration	POST	нет
Пользователь (клиент)	UC 1.1 Посмотреть список ресторанов	/restaurants	GET	нет
Пользователь (клиент)	UC 1.2 Посмотреть меню ресторана	/restaurants/{id}/dishes	GET	нет
Пользователь (клиент)	UC 1.3 Посмотреть детали блюда	/dishes/{id}	GET	нет
Пользователь	UC 2.1.1	/carts/current	GET	да

ь (клиент)	Посмотреть свою корзину			
Пользователь (клиент)	UC 2.1.2 Добавить блюдо в корзину	/cart-items	POST	да
Пользователь (клиент)	UC 2.1.3 Изменить количество в корзине	/cart-items/{id}	PATCH/PUT	да
Пользователь (клиент)	UC 2.1.4 Удалить позицию из корзины	/cart-items/{id}	DELETE	да
Пользователь (клиент)	UC 2.2 Оформить заказ	/orders	POST	да
Пользователь (клиент)	UC 2.3 Посмотреть список своих заказов	/orders	GET	да
Пользователь (клиент)	UC 2.4 Посмотреть детали заказа	/orders/{id}	GET	да
Курьер	UC 3.1 Посмотреть назначенные доставки	/deliveries/my	GET	да
Курьер	UC 3.2 Обновить статус доставки	/deliveries/{id}	PATCH	да
Администратор	UC 4.1 Добавить ресторан	/restaurants	POST	да
Администратор	UC 4.2 Добавить блюдо в меню	/dishes	POST	да
Администратор	UC 4.3 Обновить данные ресторана	/restaurants/{id}	PUT/PATCH	да

3. Добавить описание маршрутов и конечных точек в документацию проекта в вики и github pages.

№	Маршрут	HTTP-метод	Аутентификация	Описание
1	/login	GET	нет	Показать форму

				входа пользователя.
2	/login	POST	нет	Проверить логин и пароль, выдать токен/создать сессию.
3	/registration	GET	нет	Показать форму регистрации нового пользователя.
4	/registration	POST	нет	Принять данные регистрации и создать запись в users.
5	/restaurants	GET	нет	Вернуть список доступных ресторанов.
6	/restaurants/{id}/dishes	GET	нет	Вернуть меню (список блюд) выбранного ресторана.
7	/dishes/{id}	GET	нет	Вернуть подробную информацию о конкретном блюде.
8	/carts/current	GET	да	Показать текущую корзину авторизованного пользователя.
9	/cart-items	POST	да	Добавить новое блюдо в корзину (создать позицию в cart_items).
10	/cart-items/{id}	PUT / PATCH	да	Изменить количество/параметры существующей позиции корзины.
11	/cart-items/{id}	DELETE	да	Удалить позицию из корзины пользователя.
12	/orders	POST	да	Оформить заказ на основе текущей корзины, создать orders и order_items.
13	/orders	GET	да	Вернуть список заказов текущего пользователя.
14	/orders/{id}	GET	да	Вернуть детали конкретного заказа (статус, позиции, суммы).
15	/deliveries/my	GET	да	Показать курьеру список назначенных

				ему доставок.
1 6	/deliveries/{id}	PATCH	да	Обновить статус доставки (в пути, доставлено и т.п.).
1 7	/restaurants	POST	да (админ)	Добавить новый ресторан в систему.
1 8	/dishes	POST	да (админ)	Добавить новое блюдо в меню ресторана.
1 9	/restaurants/{id}	PUT / PATCH	да (админ)	Обновить данные ресторана (название, кухня, статус партнёрства и пр.).

Маршруты API

Аутентификация

- **Форма входа**
Метод: GET
Маршрут: GET /login
Описание: Показать форму входа пользователя.
- **Вход (аутентификация)**
Метод: POST
Маршрут: POST /login
Описание: Проверить логин и пароль, выдать JWT-токен / создать сессию.
- **Форма регистрации**
Метод: GET
Маршрут: GET /registration
Описание: Показать форму регистрации нового пользователя.
- **Регистрация**
Метод: POST
Маршрут: POST /registration
Описание: Принять данные регистрации и создать запись в `users`.

Рестораны и меню

- **Список ресторанов**
Метод: GET
Маршрут: GET /restaurants
Описание: Вернуть список доступных ресторанов.
- **Меню ресторана**
Метод: GET
Маршрут: GET /restaurants/{id}/dishes
Описание: Вернуть меню (список блюд) выбранного ресторана.
- **Детали блюда**
Метод: GET
Маршрут: GET /dishes/{id}
Описание: Вернуть подробную информацию о конкретном блюде.

Корзина

- **Текущая корзина**
Метод: GET
Маршрут: GET /carts/current
Аутентификация: да
Описание: Показать текущую корзину авторизованного пользователя.
- **Добавить блюдо в корзину**
Метод: POST
Маршрут: POST /cart-items
Аутентификация: да
Описание: Добавить новое блюдо в корзину (создать позицию в `cart_items`).
- **Обновить позицию в корзине**
Метод: PUT / PATCH
Маршрут: PUT /cart-items/{id}
Аутентификация: да
Описание: Изменить количество/параметры существующей позиции корзины.
- **Удалить позицию из корзины**
Метод: DELETE
Маршрут: DELETE /cart-items/{id}
Аутентификация: да
Описание: Удалить позицию из корзины пользователя.

Заказы

- **Создать заказ**
Метод: POST
Маршрут: POST /orders
Аутентификация: да
Описание: Оформить заказ на основе текущей корзины, создать записи в `orders` и `order_items`.
- **Список заказов пользователя**
Метод: GET
Маршрут: GET /orders
Аутентификация: да
Описание: Вернуть список заказов текущего пользователя.
- **Детали заказа**
Метод: GET
Маршрут: GET /orders/{id}
Аутентификация: да
Описание: Вернуть детали конкретного заказа (статус, позиции, суммы).

Доставка

- **Доставки курьера**
Метод: GET
Маршрут: GET /deliveries/my
Аутентификация: да
Описание: Показать курьеру список назначенных ему доставок.
- **Обновить статус доставки**
Метод: PATCH
Маршрут: PATCH /deliveries/{id}
Аутентификация: да
Описание: Обновить статус доставки (в пути, доставлено и т.п.).

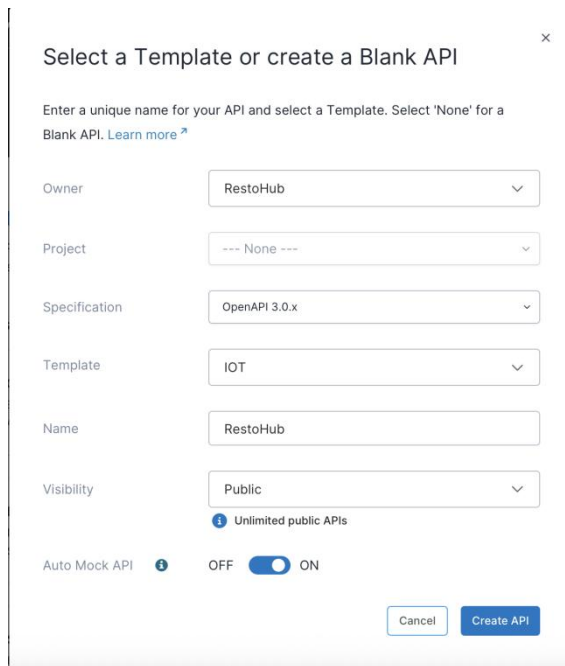
Админ

- **Добавить ресторан**
Метод: POST
Маршрут: POST /restaurants
Аутентификация: да (админ)
Описание: Добавить новый ресторан в систему.
- **Добавить блюдо**
Метод: POST
Маршрут: POST /dishes
Аутентификация: да (админ)
Описание: Добавить новое блюдо в меню ресторана.
- **Обновить данные ресторана**
Метод: PUT / PATCH
Маршрут: PUT /restaurants/{id}
Аутентификация: да (админ)
Описание: Обновить данные ресторана (название, кухня, статус партнёрства и др.).

Задание 4. Проектирование и тестирование API

1. Изучить примеры запросов из «[Swagger Petstore - OpenAPI 3.0](#)» и примеры из статьи в задании 1.

2. Создать учетную запись на сервисе с поддержкой Open API, например Swagger UI (<https://app.swaggerhub.com/home>) или Postman (<https://www.postman.com>).
3. Создать API, например на SwaggerHub, с нуля или из шаблона, или импортировав существующее API согласно статье <https://support.smartbear.com/swaggerhub/docs/en/manage-apis/creating-a-new-api.html>. Если при создании активирован переключатель для создания Mock-сервера, то он будет автоматически создан. Mock-сервер применяется для симуляции ответов.



Select a Template or create a Blank API

Enter a unique name for your API and select a Template. Select 'None' for a Blank API. [Learn more](#)

Owner: RestoHub

Project: --- None ---

Specification: OpenAPI 3.0.x

Template: IOT

Name: RestoHub

Visibility: Public

Unlimited public APIs

Auto Mock API: OFF ON

Cancel Create API

4. Определить тело для каждого POST и PUT-запроса, а также статусы HTTP ответов и их смысл в спецификации Open API как на рисунке 5.

RestoHub API

1.0.0OAS 3.0

API для сервиса доставки еды (лабораторная работа).

Servers

https://api.example.com - Основной сервер (для ...

Authorize

default

GET

/login

Форма логина

POST

/login

Аутентификация

GET

/registration

Форма регистрации

POST

/registration

Регистрация пользователя

GET

/restaurants

Список ресторанов

POST

/restaurants

Добавить ресторан (админ)

GET

/restaurants/{id}/dishes

Меню ресторана

GET

/carts/current

Текущая корзина пользователя

POST

/cart-items

Добавить блюдо в корзину

GET

/orders

Список заказов пользователя

POST

/orders

Оформить заказ

GET

/orders/{id}

Детали заказа

5. Протестировать разработанную спецификацию. Если Mock-сервер не был активирован в пункте 3 текущего задания, то настройте его согласно документации <https://support.smartbear.com/swaggerhub/docs/en/integrations/api-auto-mocking.html>

Parameters

Cancel

No parameters

Execute

Clear

Responses

Snippets

cURL (bash)

cURL (PowerShell)

cURL (CMD)

```
curl -X 'GET' \
  'https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0/login' \
  -H 'accept: */*'

```

Request URL

https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0/login

Server response

Code

Details

200

Response headers

```

access-control-allow-credentials: true
access-control-allow-headers: X-Requested-With, Content-Type, Accept, Origin
access-control-allow-methods: *
access-control-allow-origin: *
cache-control: no-cache
content-encoding: gzip
content-length: 20
content-type: application/json; charset=utf-8
date: Sat, 22 Nov 2025 16:08:56 GMT
etag: W/"14-yIMMzzqGPKico39NpXK600BdB3s"
expires: -1
server: nginx
strict-transport-security: max-age=3.1536e+07; includeSubDomains
x-powered-by: Express

```

Request duration

288 ms

Responses

Code

Description

Links

200

Страница или данные формы логина.

No links

6. Добавить примеры запросов и ссылку на документацию API в документацию проекта в вики и github pages

№	Маршрут	HT TP- мет од	Аутенти фикация	Пример запроса	Описа ние
1	/restaurants	GET	нет	GET /restaurants HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0	Возвращает список доступных ресторанов.
2	/restaurants/{id}/dishes	GET	нет	GET /restaurants/1/dishes HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0	Возвращает список блюд выбранного ресторана.

3	/carts/current	GET	да (JWT)	GET /carts/current HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>	Возвращает текущую корзину у авторизованного пользователя.
4	/cart-items	POST	да (JWT)	POST /cart-items HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>Content-Type: application/json{"dish_id": 42, "quantity": 2}	Добавляет выбранное блюдо в корзину пользователя.
5	/cart-items/{id}	PUT	да (JWT)	PUT /cart-items/10 HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>Content-Type: application/json{"quantity": 3}	Изменяет количество блюда в указанной позиции корзины.
6	/cart-items/{id}	DELETE	да (JWT)	DELETE /cart-items/10 HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>	Удаляет позицию из корзины пользователя.
7	/orders	POST	да (JWT)	POST /orders HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>Content-Type: application/json{"address_id": 5, "order_type": "delivery", "payment_method": "card"}	Создает новый заказ на основе текущей корзины

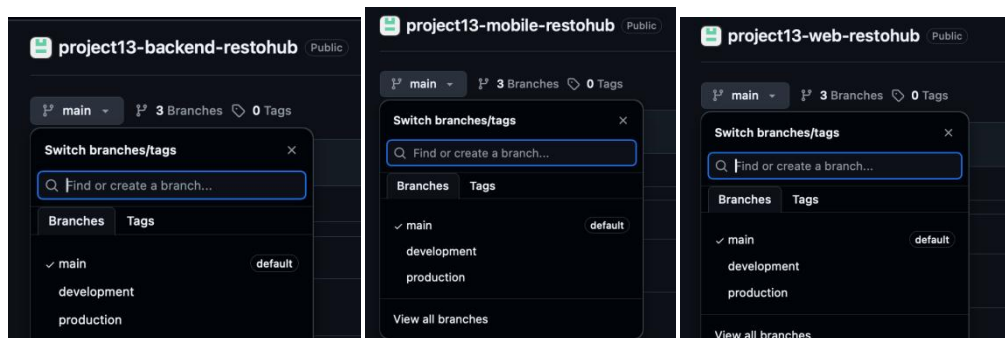
					ы.
8	/orders	GET	да (JWT)	GET /orders HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>	Возвращает список всех заказов в текущего пользователя.
9	/orders/{id}	GET	да (JWT)	GET /orders/100 HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Authorization: Bearer <token>	Возвращает детали конкретного заказа.
10	/login	POST	нет	POST /login HTTP/1.1Host: https://virtserver.swaggerhub.com/restohub/RestoHub/1.0.0Content-Type: application/json{"email": "user@example.com", "password": "string"}	Проверяет логин и пароль, возвращает JWT-токен.

Примеры запросов и ссылку на документацию API

№	Маршрут	HTTP-метод	Аутентификация	Пример запроса
1	<code>/restaurants</code>	GET	нет	<pre>GET /restaurants HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu</pre>
2	<code>/restaurants/{id}/dishes</code>	GET	нет	<pre>GET /restaurants/1/dishes HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu</pre>
3	<code>/carts/current</code>	GET	да (JWT)	<pre>GET /carts/current HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token></pre>
4	<code>/cart-items</code>	POST	да (JWT)	<pre>POST /cart-items HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token> Content-Type: application/json {"dish_id": 42, "quantity": 2}</pre>
5	<code>/cart-items/{id}</code>	PUT	да (JWT)	<pre>PUT /cart-items/10 HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token> Content-Type: application/json {"quantity": 3}</pre>
6	<code>/cart-items/{id}</code>	DELETE	да (JWT)	<pre>DELETE /cart-items/10 HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token></pre>
7	<code>/orders</code>	POST	да (JWT)	<pre>POST /orders HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token> Content-Type: application/json {"address_id": 5, "order_type": "delivery", "payment_method": "card"}</pre>
8	<code>/orders</code>	GET	да (JWT)	<pre>GET /orders HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token></pre>
9	<code>/orders/{id}</code>	GET	да (JWT)	<pre>GET /orders/100 HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Authorization: Bearer <token></pre>
10	<code>/login</code>	POST	нет	<pre>POST /login HTTP/1.1 Host: https://virtserver.swaggerhub.com/restohu Content-Type: application/json {"email": "user@example.com", "password":</pre>

Задание 5. Спроектировать инфраструктуру проекта и развернуть ее

- Изучить статью и последовательность шагов, необходимых для построения окружения разработки
<https://habr.com/ru/company/southbridge/blog/329262/> или
<https://selectel.ru/blog/tutorials/develop-canban-board-on-django-drf-alpine-js/>
- Создать структуру проекта в репозиториях для **бекенд**, **веб** и **мобильной версии** проекта на github. Создать ветки для релиз-версии (production) и тестовой версии (development). Согласовать правила работы с ветками. Разработку проекта вести только с использованием системы контроля версий.



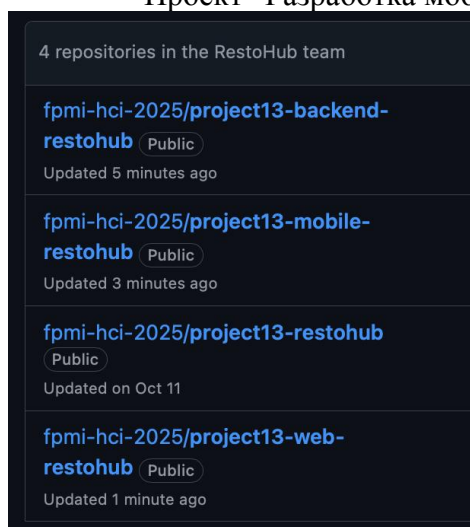
3. Управление проектом вести с использованием Project в github. Для коммуникации использовать Slack или иной мессенджер для командной работы. При необходимости создать необходимое количество проектов в основном репозитории, например:

Проект “Дизайн и проектирование приложений” – общая документация по проекту и отчеты по лабораторным работам

Проект “Разработка API и бизнес-логики”

Проект “Разработка веб-приложения”

Проект “Разработка мобильного приложения”



4. На основе технического задания, разработанного в п. 1 данной лабораторной работы, составить план работ по бекенду, включая разработку API, распределив задачи между участниками команды.

Этап / Задача	Участник А	Участник В	Участник С
Настройка репо, базовый проект, общие зависимости	ведёт	помогает	помогает
Проработка и фиксация ER-диаграммы, схемы БД			
Реализация моделей БД и миграций			
Модуль авторизации	основной		

и пользователей (/registration, /login, JWT)			
Middleware проверки токена, роли пользователя			
Модуль ресторанов и меню (/restaurants, /restaurants/{id}/dishes, админские POST)	основной		
Наполнение тестовыми ресторанами и блюдами			
Модуль корзины (/carts/current, /cart-items)		помогает	
Модуль заказов (/orders, /orders/{id})	основной		
Логика формирования заказа из корзины			
Синхронизация кода с OpenAPI на SwaggerHub (обновление спецификации)			(совместно)
Написание юнит-тестов	авторизация	рестораны/меню	корзина/заказы
Интеграционные тесты (Postman, сценарии end-to-end)	ведёт	помогает	
Подготовка докер-файла и инструкции по развёртыванию	ведёт		
Подготовка отчётной документации по бэкенду	структура, общая часть	описание ресторанов/меню	описание корзины/заказов, деплой

5. План работ добавить в проекты в системе контроля версий в основной репозиторий и документировать проект в Readme, wiki и Github Pages репозитория согласно требованиям, представленным в Документирование

проекта.

План работ по бэкенду и распределение задач			
Этап / Задача	Участник А	Участник В	Участник С
Настройка реп, базовый проект, общие зависимости	✓ ведёт	помогает	помогает
Проработка и фиксация ER-диаграммы, схемы БД	✓ ведёт	✓	
Реализация моделей БД и миграций	✓	✓	
Модуль авторизации и пользователей (/authentication, /login, /jwt)	✓ основной		
Модуль проверки токена, роли пользователя	✓		
Модуль ресторанов и меню (/restaurants, /restaurants/{id}/details, /restaurants/POST)	✓ основной		
Наполнение тестовыми ресторанами и блюдами	✓		
Модуль корзины (/carts/current, /cart-items)	✓	✓ помогает	
Модуль заказов (/orders, /orders/{id})	✓ основной		
Логика формирования заказа из корзины	✓		
Синхронизация кода с OpenAPI на SwaggerHub (обновления спецификации)	✓	✓	✓ (совместно)
Написание юнит-тестов	авторизация	рестораны/меню	корзина/заказы
Интеграционные тесты (Postman, сценарии end-to-end)	✓ ведёт	✓ помогает	
Подготовка докер-файла и инструкции по развертыванию	✓ ведёт		
Подготовка отчётной документации по бэкенду	структура, общая часть	описание ресторанов/заказов, деталей	описание корзины/меню

6. Добавить автоматические тесты, чтобы сервер CI перед сборкой выполнял тесты и формировал сборку только после их прохождения.

```
1  name: CI/CD Pipeline
2
3  on:
4    push:
5      branches:
6        - main
7    pull_request:
8      branches:
9        - main
10
11  jobs:
12    test-and-build:
13      runs-on: ubuntu-latest
14
15      steps:
16        - name: Checkout repository
17          uses: actions/checkout@v2
18
19        - name: Set up Node.js
20          uses: actions/setup-node@v2
21          with:
22            node-version: '16'
23
24        - name: Install dependencies
25          run: npm install
26          working-directory: ./backend
27
28        - name: Run tests
29          run: npm test
30          working-directory: ./backend
31
32        - name: Build application
33          if: success()
34          run: npm run build
35          working-directory: ./backend
```

project13-backend-restohub / backend / package.json

polina0710 Make build step dummy ✓

Code Blame 32 lines (32 loc) · 693 Bytes

```
1  {
2    "name": "RestoHub",
3    "version": "1.0.0",
4    "description": "Backend API for the RestoHub application, managing orders, restaurants and user interactions.",
5    "main": "src/index.js",
6    "scripts": {
7      "start": "node src/index.js",
8      "test": "jest",
9      "build": "echo \\\"Build step (no real build)\\\""
10   },
11   "dependencies": {
12     "express": "^4.17.1",
13     "mongoose": "^5.12.0"
14   },
15   "devDependencies": {
16     "jest": "^26.6.3",
17     "supertest": "^6.1.3"
18   },
19   "repository": {
20     "type": "git",
21     "url": "https://github.com/YourOrg/RestoHub-backend.git"
22   },
23   "keywords": [
24     "restohub",
25     "restaurant",
26     "api",
27     "node",
28     "backend"
29   ],
30   "author": "Girls",
31   "license": "ISC"
32 }
```

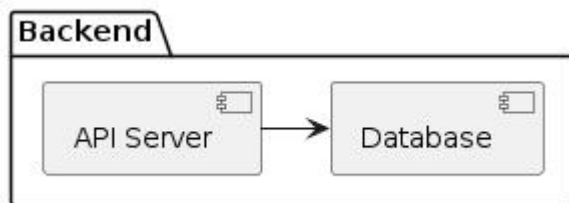
7. Спроектировать инфраструктуру бекенд и веб-приложения на основе системы контейнеризации Docker инфраструктуру, используя PlanUML. Если диаграммы спроектирована в лабораторной работе 3, уточнить ее, при необходимости.

1. Настроим контейнер для backend

- Цель: Этот контейнер будет содержать API сервер, обрабатывать запросы от клиента, управлять данными и взаимодействовать с базой данных.

Обозначение в PlanUML:

```
package Backend {  
  [API Server] -right-> [Database]  
}
```



2. Настроим контейнер базы данных

- Цель: Разделить базу данных в отдельный контейнер, чтобы гарантировать безопасность и лёгкость управления.

Обозначение в PlanUML:

```
package Database {  
  [Database: PostgreSQL/MySQL]  
}
```

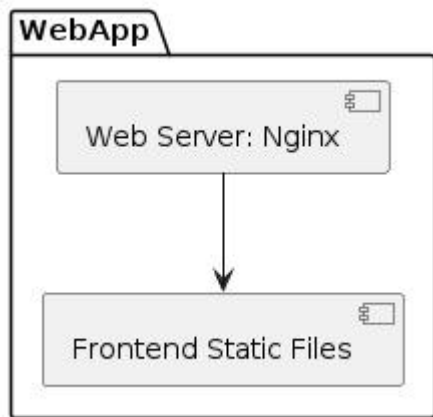


3. Настроим контейнер для web-приложения

- Цель: Веб-сервер (например, Nginx) будет обслуживать статические файлы и обеспечивать поддержку клиентских запросов.

Обозначение в PlanUML:

```
package WebApp {  
  [Web Server: Nginx] --> [Frontend Static Files]  
}
```

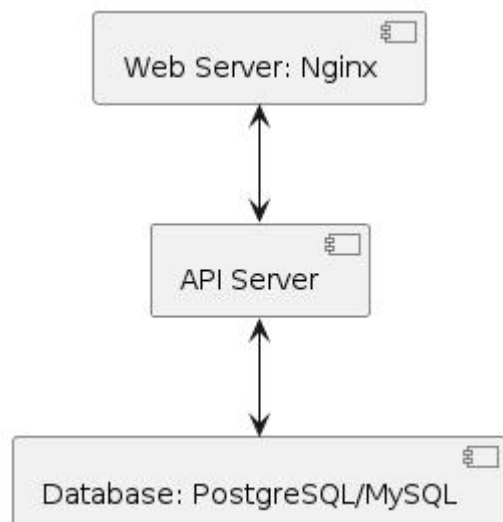
4. Настроим сеть Docker

- Цель: Связь между backend, web-приложением и базой данных через сеть Docker для внутренних коммуникаций между контейнерами.

Обозначение в PlanUML:

[API Server] <--> [Database: PostgreSQL/MySQL]

[Web Server: Nginx] <--> [API Server]



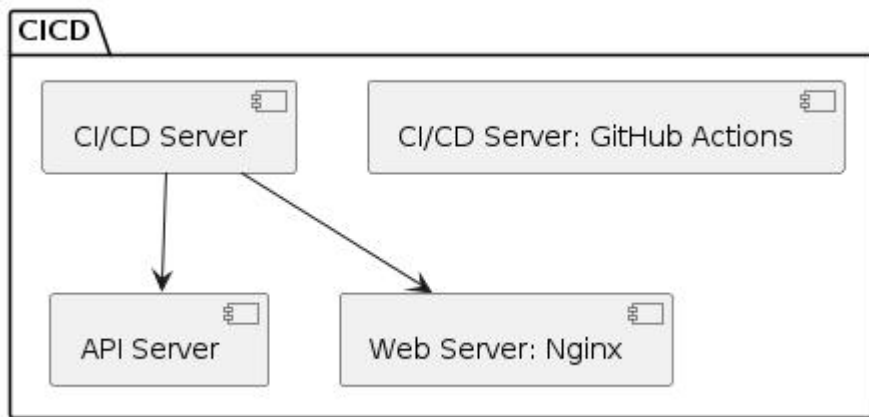
5. Настроим CI/CD сервер

- Цель: Сервер CI/CD (например, GitHub Actions или Jenkins) будет автоматически тестировать и развертывать контейнеры.

Обозначение в PlanUML:

```

package CICD {
    [CI/CD Server: GitHub Actions]
    [CI/CD Server] -down-> [API Server]
    [CI/CD Server] -down-> [Web Server: Nginx]
}
    
```



6. Добавляем логирование и мониторинг

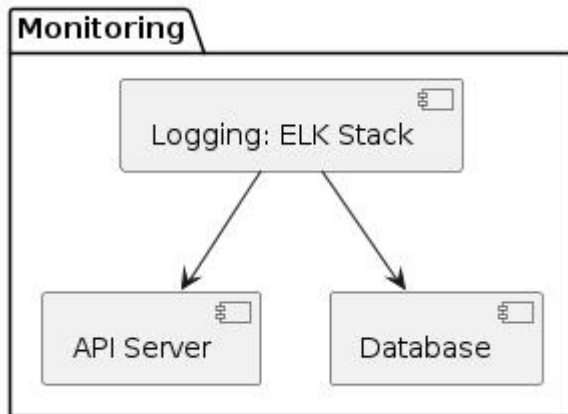
- Цель: ELK Stack или другие инструменты для логирования и мониторинга обеспечат наблюдение за работой контейнеров.

Обозначение в PlanUML:

```

package Monitoring {
    [Logging: ELK Stack] --> [API Server]
    [Logging: ELK Stack] --> [Database]
}

```



7. Оркестрация контейнеров

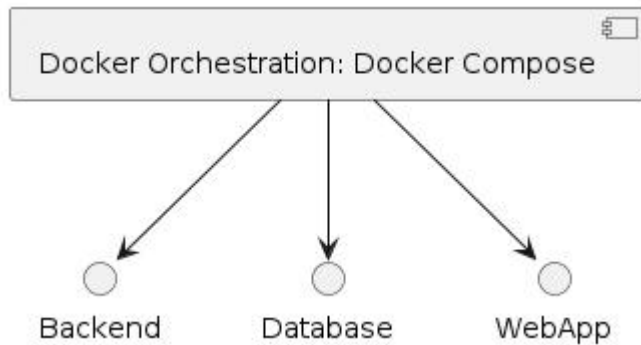
- Цель: Использовать Docker Compose или Docker Swarm для управления и развертывания нескольких контейнеров.

Обозначение в PlanUML:

```

[Docker Orchestration: Docker Compose] --> Backend
[Docker Orchestration: Docker Compose] --> Database
[Docker Orchestration: Docker Compose] --> WebApp

```



Итоговая диаграмма PlanUML

Объединив всё вышеуказанное, можно представить итоговую инфраструктурную диаграмму следующим образом:

@startuml

```

package Backend {
  [API Server] -right-> [Database]
}
  
```

```

package Database {
  [Database: PostgreSQL/MySQL]
}
  
```

```

package WebApp {
  [Web Server: Nginx] --> [Frontend Static Files]
}
  
```

```

package CICD {
  [CI/CD Server: GitHub Actions]
  [CI/CD Server] -down-> [API Server]
  [CI/CD Server] -down-> [Web Server: Nginx]
}
  
```

```

package Monitoring {
  [Logging: ELK Stack] --> [API Server]
  [Logging: ELK Stack] --> [Database]
}
  
```

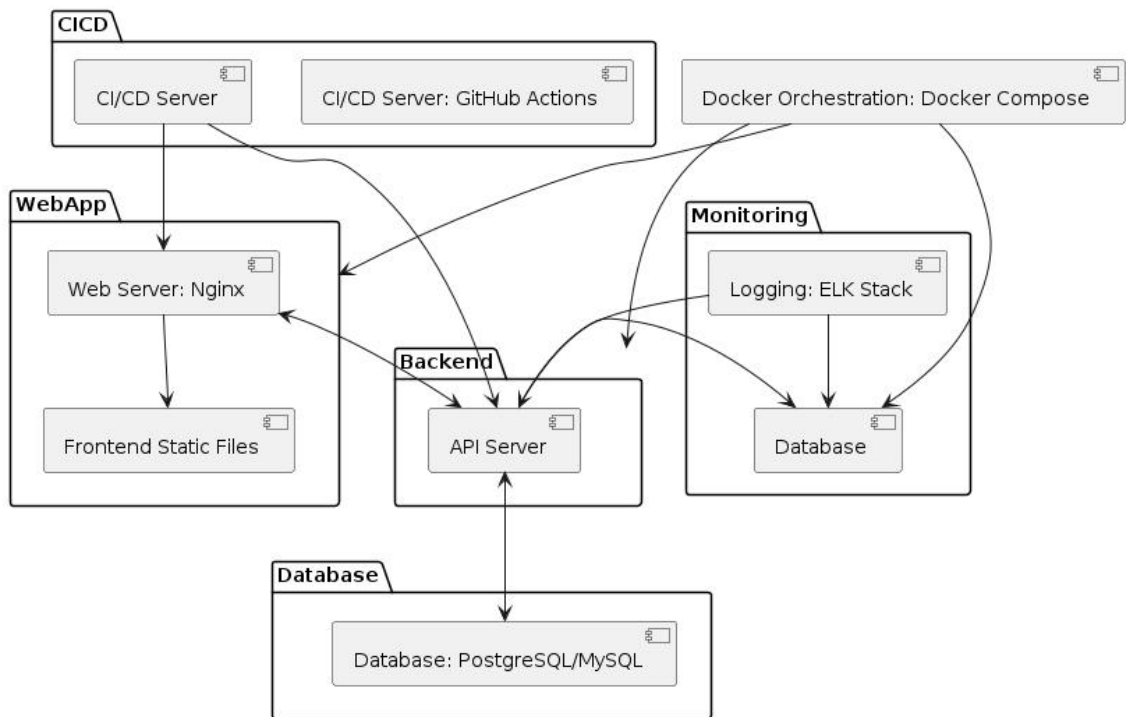
```

[API Server] <--> [Database: PostgreSQL/MySQL]
[Web Server: Nginx] <--> [API Server]
  
```

```

[Docker Orchestration: Docker Compose] --> Backend
[Docker Orchestration: Docker Compose] --> Database
[Docker Orchestration: Docker Compose] --> WebApp
  
```

@enduml



8. Развернуть инфраструктуру для бекенд, веб-приложения на основе контейнеров, конфигурацию которых описать в конфигурационных файлах Dockerfile и для запуска использовать Docker Compose.

project13-backend-restohub / backend / Dockerfile

polina0710 Add dockerfile

Code Blame 12 lines (7 loc) · 111 Bytes

```

1 FROM node:18
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 EXPOSE 5000
11
12 CMD ["npm", "start"]

```

project13-web-restohub / Dockerfile

polina0710 Add dockerfile

Code Blame 14 lines (8 loc) · 129 Bytes

```

1 FROM node:18
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 RUN npm run build
11
12 EXPOSE 3000
13
14 CMD ["npm", "start"]

```

project13-web-restohub / docker-compose.yml

polina0710 Add dockerfile

Code Blame 13 lines (12 loc) · 225 Bytes

```

1 version: "3.9"
2
3 services:
4   web:
5     build:
6       context: .
7       dockerfile: Dockerfile
8     container_name: restohub-web
9     ports:
10      - "3000:3000"
11     environment:
12       NODE_ENV: production
13     restart: unless-stopped

```

project13-backend-restohub / backend / docker-compose.yml

polina0710 Add dockerfile

Code Blame 14 lines (12 loc) · 238 Bytes

```

1 version: "3.9"
2
3 services:
4   backend:
5     build:
6       context: .
7       dockerfile: Dockerfile
8     container_name: restohub-backend
9     ports:
10      - "5000:5000"
11     environment:
12       NODE_ENV: production
13     restart: unless-stopped
14

```

9. Добавить простое приложение типа «Hello World» для иллюстрации работоспособности каждого настроенного окружения.

CodeBlame

11 lines (9 loc) · 255 Bytes

```
1  const express = require('express');
2  const app = express();
3
4  app.get('/', (req, res) => {
5    res.send('Hello, World!');
6  });
7
8  const PORT = process.env.PORT || 3000;
9  app.listen(PORT, () => {
10    console.log(`Server is running on http://localhost:${PORT}`);
11  });
```

CodeBlame

11 lines (11 loc) · 175 Bytes

```
1  {
2    "name": "hello-world-app",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "start": "node index.js"
7    },
8    "dependencies": {
9      "express": "^4.17.1"
10   }
11 }
```

Code**Blame**

18 lines (13 loc) · 556 Bytes

```
1  # Используем официальный образ Node.js как базовый
2  FROM node:14
3
4  # Устанавливаем рабочую директорию в контейнере
5  WORKDIR /app
6
7  # Копируем package.json и устанавливаем зависимости
8  COPY package.json ./
9  RUN npm install
10
11 # Копируем весь код приложения
12 COPY . .
13
14 # Указываем, что контейнер будет слушать на порту 3000
15 EXPOSE 3000
16
17 # Запускаем сервер
18 CMD ["npm", "start"]
```

Code**Blame**

2 lines (2 loc) · 27 Bytes

```
1  node_modules
2  npm-debug.log
```

Code**Blame**

32 lines (25 loc) · 668 Bytes

```
1   name: CI/CD Pipeline
2
3   on:
4     push:
5       branches:
6         - main
7
8   jobs:
9     build:
10      runs-on: ubuntu-latest
11
12      steps:
13      - name: Checkout code
14        uses: actions/checkout@v2
15
16      - name: Set up Docker
17        uses: docker/setup-buildx-action@v2
18
19      - name: Log in to Docker Hub
20        uses: docker/login-action@v2
21        with:
22          username: ${ secrets.DOCKER_USERNAME }
23          password: ${ secrets.DOCKER_PASSWORD }
24
25      - name: Install Docker Compose
26        run: |
27          sudo apt-get update
28          sudo apt-get install -y docker-compose
29
30      - name: Build and run containers with Docker Compose
31        run: |
32          docker-compose -f docker-compose.yml up -d
```

```
Code Blame 39 lines (30 loc) - 886 Bytes

1  name: Docker Build and Push
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11
12     steps:
13     - name: Checkout code
14       uses: actions/checkout@v2
15
16     - name: Set up Docker Buildx
17       uses: docker/setup-buildx-action@v2
18
19     - name: Log in to Docker Hub
20       uses: docker/login-action@v2
21       with:
22         username: ${ secrets.DOCKER_USERNAME }
23         password: ${ secrets.DOCKER_PASSWORD }
24
25     - name: Install Docker Compose
26       run: |
27         sudo apt-get update
28         sudo apt-get install -y docker-compose
29
30     - name: Build Docker image
31       run: docker build -t hello-world-app .
32
33     - name: Run Docker Compose
34       run: |
35         docker-compose up -d
36
37     - name: Verify the application is running
38       run: |
39         curl http://localhost:3000
```

Проверяем работоспособность

 Update docker.yml	main	 8 minutes ago	...
CI/CD Pipeline #10: Commit 0bd7d68 pushed by tanyashybko			
 Update hello-world.yml	main	 18 minutes ago	...
CI/CD Pipeline #8: Commit e47ede1 pushed by tanyashybko			